sensors

MDPI

Article

A Complete Coverage Path Planning Algorithm for Lawn Mowing Robots Based on Deep Reinforcement Learning

Ying Chen [1,2,†], Zhe-Ming Lu [1,2,*,†], Jia-Lin Cui [1,3,*], Hao Luo [1,2] and Yang-Ming Zheng [1,2]

[1] Center for Generic Aerospace Technology, Huanjiang Laboratory, Zhuji 311816, China; nampl@zju.edu.cn (Y.C.); luohao@zju.edu.cn (H.L.); zymsun2002@zju.edu.cn (Y.-M.Z.)
[2] School of Aeronautics and Astronautics, Zhejiang University, Hangzhou 310027, China
[3] School of Information Science and Engineering, NingboTech University, Ningbo 315100, China
* Correspondence: zheminglu@zju.edu.cn (Z.-M.L.); cuijl_jx@163.com (J.-L.C.); Tel.: +86-571-87953349 (Z.-M.L.)
† These authors contributed equally to this work.

Abstract: This paper introduces Re-DQN, a deep reinforcement learning-based algorithm for comprehensive coverage path planning in lawn mowing robots. In the fields of smart homes and agricultural automation, lawn mowing robots are rapidly gaining popularity to reduce the demand for manual labor. The algorithm introduces a new exploration mechanism, combined with an intrinsic reward function based on state novelty and a dynamic input structure, effectively enhancing the robot's adaptability and path optimization capabilities in dynamic environments. In particular, Re-DQN improves the stability of the training process through a dynamic incentive layer and achieves more comprehensive area coverage and shorter planning times in high-dimensional continuous state spaces. Simulation results show that Re-DQN outperforms the other algorithms in terms of performance, convergence speed, and stability, making it a robust solution for comprehensive coverage path planning. Future work will focus on testing and optimizing Re-DQN in more complex environments and exploring its application in multi-robot systems to enhance collaboration and communication.

Keywords: path planning; complete coverage path planning; reward function; curiosity-driven exploration; dynamic ε-greedy strategy; dynamic environments; training stability

Academic Editors: Oscar Reinoso García, Luis Payá and Helder Jesus Araújo

Received: 28 November 2024
Revised: 6 January 2025
Accepted: 8 January 2025
Published: 12 January 2025

Citation: Chen, Y.; Lu, Z.-M.; Cui, J.-L.; Luo, H.; Zheng, Y.-M. A Complete Coverage Path Planning Algorithm for Lawn Mowing Robots Based on Deep Reinforcement Learning. *Sensors* **2025**, *25*, 416. https://doi.org/10.3390/s25020416

1. Introduction

With the rapid advancement of science and technology, mobile robots have become integral to various applications, including infrastructure inspection, rescue operations, and environmental maintenance tasks such as lawn mowing. A critical aspect of these applications is the need for complete coverage path planning (CCPP), which ensures that the robot can navigate and cover every part of a designated area. This requirement is essential for tasks where thoroughness is paramount, such as cleaning, inspection, and agricultural operations. The complexity of environments and the need for optimization and adaptability to different surfaces and obstacles make this an ongoing research challenge.

Traditional complete coverage path planning methods can be broadly categorized into the following types: partition-based, graph theory-based, and artificial intelligence-based methods. Choset [1] proposed that the CCPP algorithm can be divided into two categories: "online" and "offline". The "offline" approach assumes that environmental factors are known, including the shape and area of the coverage region as well as the distribution of obstacles. In contrast, the "online" approach uses sensors equipped on the device to perform real-time scanning of the target environment when environmental information

is completely or partially unknown. In the context of CCPP, a dynamic environment can be defined as one that contains real-time changing obstacles, unpredictable terrain variations, or other environmental conditions that may affect the robot's path selection and navigation. Such an environment requires algorithms to perceive changes in real-time and adjust the path accordingly to ensure effective coverage of the target area without collisions or redundant coverage. This work specifically focuses on the "online" approach, highlighting its unique ability to adapt in real-time to dynamic environments and ensure efficient coverage under changing conditions. The subsequent sections will delve deeper into the online strategy and its practical application.

Latombe's trapezoidal decomposition [2] splits the non-obstacle area into trapezoids [3], simplifying path planning but causing redundancy and inefficiency for irregular shapes. Choset's boustrophedon method [4] reduces subregions for shorter paths [5]. As the first two use only vertical/horizontal cuts, Huang proposed the variable cutting direction method [6] to minimize robot turns.

Then came the grid method, first proposed by Elfes and Moravec [7]. Its coverage principle is to find an optimal non-repetitive path through all free grid cells. Based on it, Hodgkin and Huxley proposed the H-H model [8]. Grossberg introduced a neural dynamic network model [9]. Gabriely et al. [10] presented the spiral grid coverage method (Spiral-STC). Gonzalez et al. improved Spiral-STC by including "partially occupied" cells in the outer spiral for full area coverage [11]. Choi et al., using historical sensor data, introduced a map coordinate assignment to reduce robot turns [12].

As neural networks emerged, the field of path planning received new opportunities. Luo et al. [13] and Yang and Luo [14] used a neural net for CCPP in floor cleaning. DFS [15] and Q-learning [16] are common. DFS ignores path length, leading to long paths and inefficiency. Q-learning needs many samples and long training for good results.

A* [17] and RRT [18] are classic path planning algorithms that are widely used. However, the A* algorithm is not suitable for dynamic scenarios, and the Rapidly Exploring Random Tree (RRT) algorithm cannot guarantee an optimal path because it is heuristic. Meanwhile, the time-consuming calculations involved in high-dimensional maps [19], along with their poor generalization capabilities, have emerged as the main drawbacks for these algorithms.

With the emergence of the Deep Q Network (DQN), which combines deep learning and reinforcement learning, robotic path planning has undergone a profound transformation [20–22]. In particular, advances such as Double DQN (DDQN) [23,24] have further enhanced the capability of DQN to learn effective strategies in complex environments. These advances enable agents to navigate and plan optimal routes more efficiently in dynamic environments, addressing the shortcomings of traditional algorithms.

In dynamic environments, many studies have explored how to effectively handle real-time changing obstacles. For example, ref. [25] presented a path planning method for dynamic obstacles, which includes a reward function with penalty terms for the training process and employs a strategy of randomly setting starting and target points to increase the diversity of the training environment. Additionally, ref. [26] discussed a grid-based method suitable for obstacle localization and path planning, introducing a "shortest distance first" strategy to reduce the path length for drones reaching their targets. Ref. [27] investigated a multi-step update strategy that brings the Q network closer to the target value, which is particularly important when dealing with dynamic obstacles.

Multi-agent reinforcement learning (MARL) is an important branch of reinforcement learning (RL) and shows significant advantages in complex system tasks such as complete coverage path planning (CCPP) and searching tasks.

Wang et al. [28] automatically designed the trigger conditions for action advising based on genetic programming (GP) to optimize the collaborative decision-making of multiple agents and improve the adaptability in dynamic environments. Yuan et al.'s two-stage planning method [29] transforms the coverage path planning problem into an optimal grid selection problem, providing ideas for optimizing the path planning of a single agent. Ramezani et al. [30] constructed a fault-tolerant framework to ensure the system performance when some agents fail, emphasizing the importance of algorithm robustness.

Although this paper focuses on the path planning of a single agent, the achievements of MARL methods provide references for single-agent research in terms of collaboration, environmental adaptation, and reliability, which is helpful for expanding the single-agent algorithm to multi-agent scenarios and optimizing the path planning algorithm.

This paper introduces a novel algorithm called Re-DQN, where 'Re' stands for 'Reinforced', indicating that this algorithm is an improved and enhanced deep reinforcement learning method based on traditional DQN, which is designed to overcome the limitations of conventional path planning methods. Our Re-DQN utilizes deep reinforcement learning to enhance the efficiency, stability, and adaptability of lawn mowing robots operating in dynamic environments.

The rest of this paper is organized as follows. Section 2 describes the modeling of the robot and its environment, including considerations for obstacle avoidance and map preprocessing. Section 3 discusses the DQN algorithm and introduces the enhancements made in the Re-DQN algorithm. Section 4 compares and discusses the performance of the Re-DQN algorithm against traditional DQN. Section 5 summarizes the findings and discusses potential directions for future research.

2. Workspace Modeling and Robotic System Overview

This section first elaborates on the model for coverage path planning, which encompasses the robot model and the environment model. Then, it presents the design of the robot system, encompassing both the hardware and software platform setup. Consequently, this provides a solid foundation for subsequent research on localization and coverage algorithms.

An overview of the complete coverage path planning problem is shown in Figure 1. The objective is to generate a path that can cover all target points. The generated coverage path should be optimal to ensure a low repetition rate and high coverage efficiency.

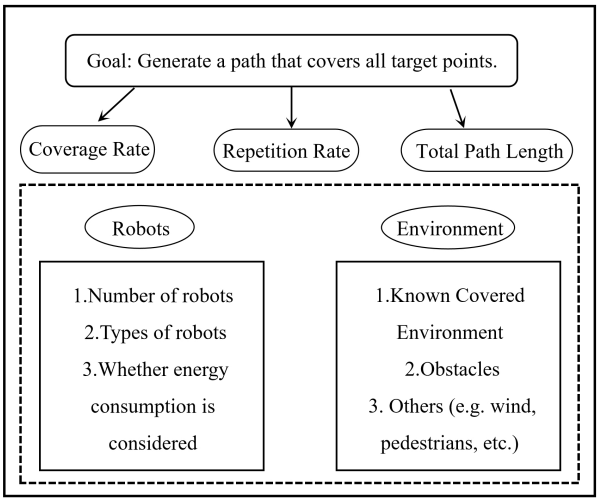

Figure 1. Overview of the CCPP problem.

2.1. Hardware and Software Architecture

Figure 2 shows the main movement components and sensor layout of the mower robot. First, the GNSS antennas on the left and right sides are used to receive GPS signals. These antennas, combined with RTK technology, provide high-precision positioning information for the mower robot.

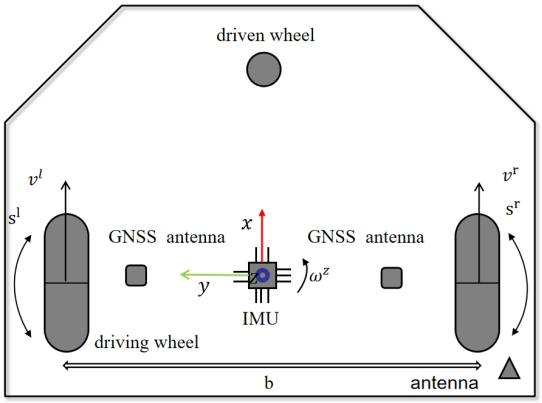

Figure 2. Key components and parameters of the robot.

The IMU, marked as a blue square in the diagram, is installed near the robot's center of rotation to measure posture changes. The robot has a front swivel caster and two rear wheels driven by independent motors. With labeled linear and rotational speeds of the wheels, and a wheel track variable b affecting turning radius and maneuverability, along with its own coordinate system, this design enables autonomous navigation and precise control in complex outdoor environments.

2.1.1. Integrated Hardware and Software System

The lawn mowing robot integrates hardware and software systems to ensure precise and autonomous operation. The hardware components include a power drive, MCU, IMU, two GNSS receivers, and UHF/VHF radio modules. The MCU is responsible for controlling the drive system and processing feedback from the wheel encoders, while the IMU and GNSS work together with RTK correction data received through the UHF/VHF module to further enhance positioning accuracy. The software architecture consists of Modbus, IMU and GNSS decoders, control modules, and positioning modules. These modules enable the real-time processing of sensor data, path planning, and accurate navigation, ensuring the robot can autonomously perform the mowing tasks along the planned route.

2.1.2. Base Station Overview

The base station of the mowing robot system undertakes the responsibilities of path planning and data uploading. It acquires environmental position data via a GNSS receiver, calculates the robot's path, and sends this information to the robot's database for the robot to follow during task execution. Moreover, the base station provides RTK correction data through the UHF/VHF radio module to improve the robot's positioning accuracy.

As shown in Figure 3, the workflow encompasses path planning, real-time positioning, navigation and control, and data feedback and correction. The base station plans the path and transmits it to the robot. The robot uses sensors like IMU and GNSS to update its position. The control module adjusts motion commands based on error and controls the drive system via the MCU for accurate path following. Meanwhile, the robot receives RTK correction signals from the base station via UHF/VHF radio for high-precision positioning.

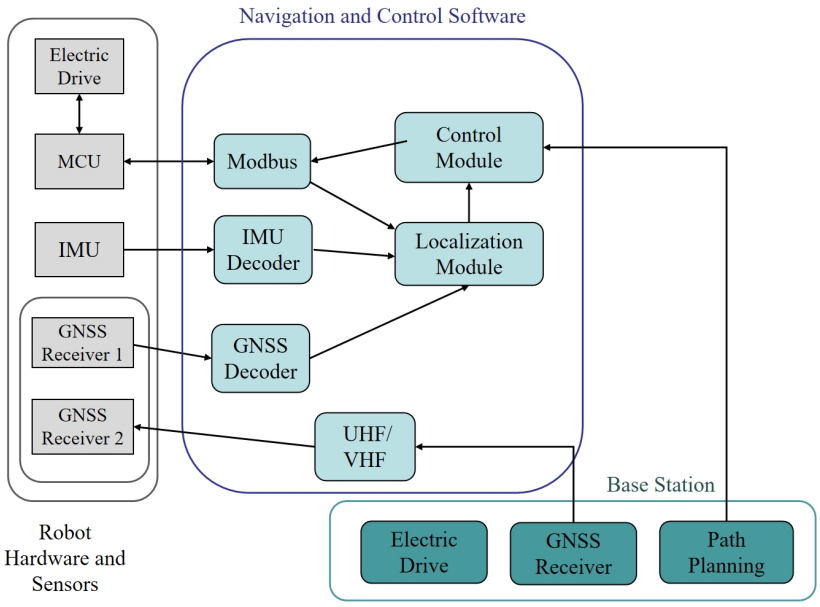

Figure 3. Overview of the system architecture.

2.2. Simplified Robot Modeling

When the lawn mowing robot navigates within its designated workspace, it is imperative to prevent collisions with obstacles or the boundaries of the map. This requires a meticulous consideration of the robot's dimensions and contours. In our work, we opt not to pursue real-time modeling for the grass-cutting robot due to concerns over high computational resource consumption and low path planning efficiency. Real-time modeling involves continuously updating the robot's state and environmental information, which can demand significant computational resources, particularly in complex environments and large-scale work areas. Therefore, to avoid the aforementioned drawbacks, we opt to inflate obstacles to the size of the grass-cutting robot, ensuring their width equals the robot's radius [31,32]. The obstacle cells in the map are enlarged by the robot's radius r_r, which is defined as the distance from the robot's center to its furthest perimeter point. For additional safety, the obstacle cells are actually enlarged by a radius $r_{obs} = r_r + d_{min}$, where d_{min} is the minimum distance between the robot and the obstacle. By enlarging the obstacles to r_{obs}, the robot can be treated as a point-like vehicle.

This approach simplifies the irregular shape of the grass-cutting robot into a circular form as shown in Figure 4. Consequently, when calculating paths near the inflated obstacles, the algorithm ensures that the robot's circular model does not intersect with any obstacles, thereby guaranteeing collision-free movement of the grass-cutting robot [33].

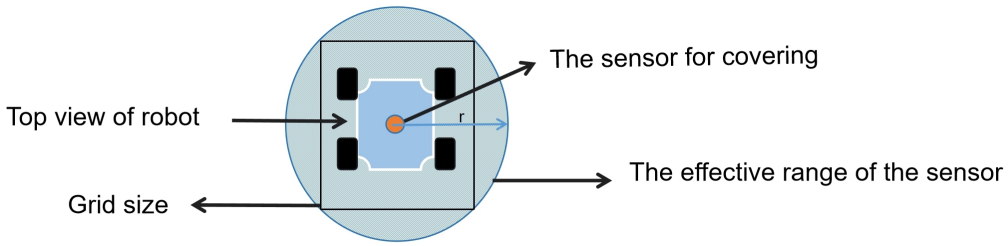

Figure 4. The relationship between the grid size and the sensor range.

Designing the agent as a circle has multiple advantages: Firstly, collision detection is simplified. That is, when calculating collisions between the agent and obstacles, it is only necessary to compare the distance from the agent's center to the obstacle's boundary to see

if it is less than or equal to the radius. This collision detection algorithm is more efficient compared to complex shapes. Secondly, coverage efficiency is high. That is, in CCPP, a circular agent can move and rotate more smoothly, effectively covering the area and reducing redundant coverage and omissions. Lastly, calculations are simplified. That is, to determine if a point is within the agent's coverage area, it is only necessary to calculate the distance from the point to the agent's center and compare it to the radius, making geometric calculations more straightforward.

2.3. Workspace Modeling and Preprocessing

In CCPP, the resolution of the grid map is a crucial factor affecting algorithm efficiency and coverage accuracy. To balance coverage accuracy and computational efficiency, this paper adopts a strategy based on the size of the agent: the resolution of each grid cell matches the coverage area of the agent, meaning each grid cell represents the area the agent can cover in one move. By setting an appropriate agent size, we ensure that the path planning precision is maintained while optimizing the use of computational resources.

The agent size directly influences the size of the grid cells and the resolution of the map. A smaller agent_size means higher resolution, providing more detail, while a larger agent_size means lower resolution, reducing memory usage. This method ensures that the grid map provides sufficient environmental details while optimizing memory usage and computational efficiency.

Before implementing the CCPP for the lawn mower robot, it is essential to observe the area that the robot needs to cover. In the grid map, the general area that the lawn mower robot needs to cover consists of all the blank grids. However, there are some special cases. For example, as shown in Figure 5, if a blank grid is surrounded by obstacles, the lawn mower robot cannot enter the blank grid for coverage.

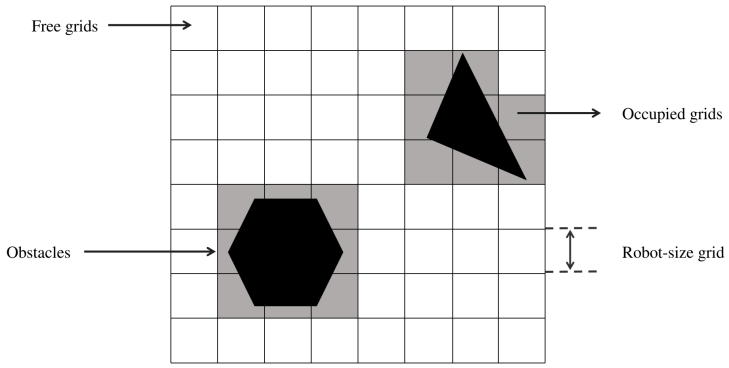

Figure 5. Environment segmentation into navigable and obstructed grids. The polygons and triangles represent actual obstacles, and the gray areas indicate the places that have already been occupied.

Therefore, it is necessary to address this situation, whereby the obstructed blank grids on the map become grids with obstacles. This implies that if a grid cannot be covered due to the presence of obstacles around it, the grid will be considered to have obstacles, thus affecting the path planning process as shown in Figure 6.

For computers, maps are a series of two-dimensional matrix inputs. Firstly, it is necessary to convert the grid map into a two-dimensional state matrix form, where the elements representing empty grids are set to 0, the elements representing obstacle grids are set to 1, and the elements representing covered grids in the state matrix are also set to 1. When the agent appears randomly in the grid during training, its position is also marked as 1. The grid map is shown in Figure 7.

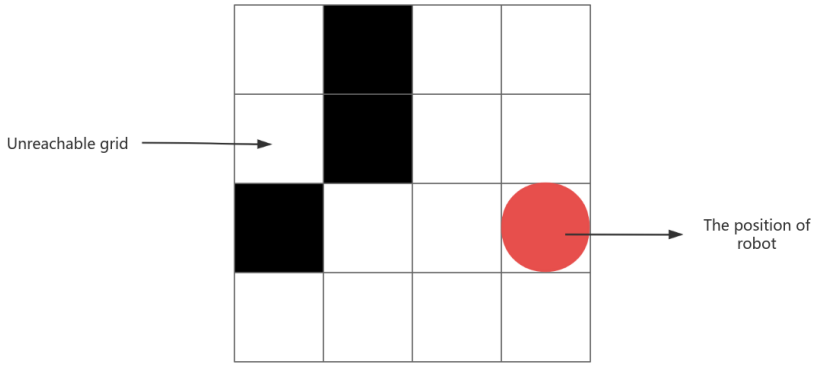

Figure 6. Unreachable grid.

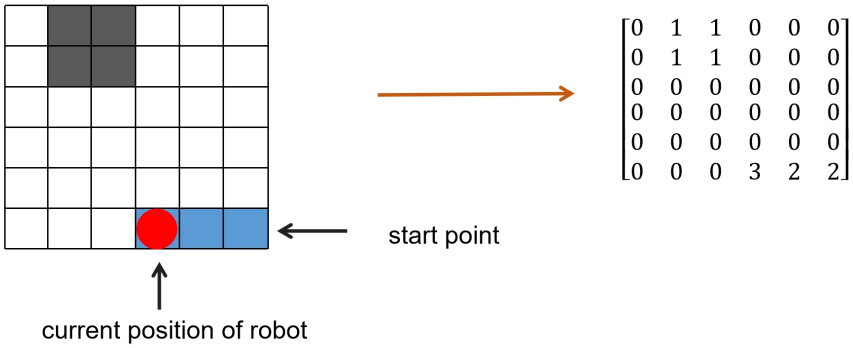

Figure 7. A state in the process. The left side shows the map, while the right side represents the state matrix in a grid mapping form.

Because the perimeter of the map serves as the map boundary and is considered an obstacle, the lawn mower robot should not exceed the map boundary. Therefore, initially, add 1 around the two-dimensional matrix representing the grid map as shown in Equation (1):

$$\begin{bmatrix} 1.0 & 1.0 & 1.0 & 1.0 & 1.0 \\ 1.0 & 0.0 & 0.0 & 0.0 & 1.0 \\ 1.0 & 0.0 & 0.0 & 0.0 & 1.0 \\ 1.0 & 1.0 & 1.0 & 1.0 & 1.0 \end{bmatrix} \quad (1)$$

In order to increase the randomness and diversity of the experiments [34], this study chooses to use newly generated maps for each episode as shown in Figure 8. The green squares in the figure represent the positions of dynamic obstacles, which move over time. The points marked as S_0, S_1,... indicate the positions of dynamic obstacles in different states. At different time points (states S_0, S_1,...,S_N), these dynamic obstacles may occupy different grid locations. The benefit of this approach is that the environment for each episode is randomly generated, which helps to evaluate the algorithm's generalization ability and robustness. Additionally, using new maps can reduce the risk of over-fitting, as the algorithm does not overly adapt to specific maps but rather needs to adapt to different environments.

This approach is more akin to real-world scenarios, as robots often need to deal with various environments and situations. Furthermore, using a new map for each episode also helps to improve the reliability and reproducibility of the experiments, as the experimental results are not influenced by specific maps, making the results more convincing and credible.

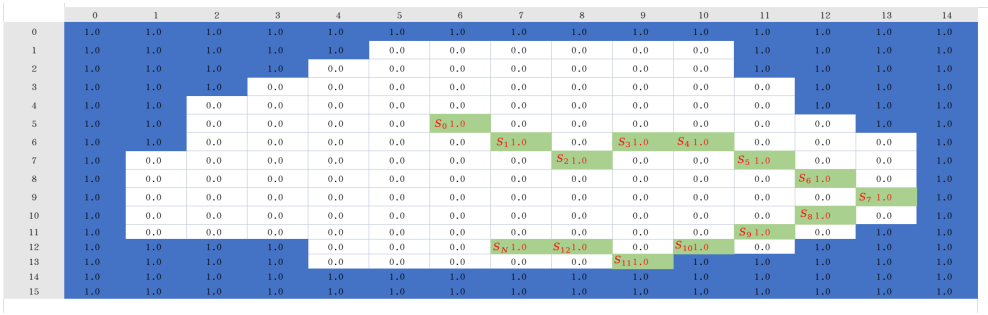

Figure 8. The random map for the N-th iteration.

Figure 9 shows a grid map containing both static and dynamic obstacles. The dynamic obstacle moves randomly in four different directions from its initial position. The green square represents the next state of the dynamic obstacle. The static obstacles (black shapes) remain stationary, while the dynamic obstacle (blue square) randomly moves to its destination (green square) in one of the directions indicated by the red arrows.

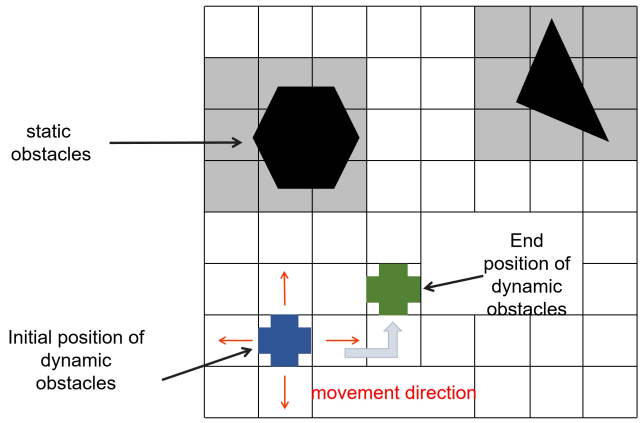

Figure 9. State representation.

A sample state environment space is illustrated as in Figure 9, the environment in this paper is unknown to robots. The robots collect information in an unknown environment through continuous exploration. S_t represents the position of the robots on the map. The action space of robots $\alpha_t = \{v_t, L\}$, $v_t \in \{up, down, left, right\}$. L is the duration of the robot's movement. In many applications, the robot is required to move at a constant speed, so the L of each step of the robot is set as a fixed value.

3. Deep Reinforcement Learning Models and the Improvements

3.1. The DQN Algorithm

The state–action function gives the basis for RL algorithms to make decisions, either using existing info (exploitation) or trying new actions (exploration). In deep RL, a deep neural net approximates this function. The parameter update formula is

$$Q(S_t, A_t, w) \leftarrow Q(S_t, A_t, w) + \alpha\left[R_{t+1} + \gamma max_a \widehat{q}(s_{t+1}, a_t, w)\right] \quad (2)$$

In Equation (2), $Q(S_t, A_t, w)$ represents the Q-value when taking action A_t in state S_t based on the parameter w. α is the learning rate that controls the update magnitude. R_{t+1} is the reward at time $t+1$. γ weighs the importance of future rewards. $max_a \hat{q}(s_{t+1}, a_t, w)$ is the maximum Q value among all actions in state s_{t+1} .

In DQN, the Q-value update method is based on the Q-learning algorithm [35] and incorporates neural networks to approximate the Q-value function. It minimizes the mean squared error between the target Q value and the current Q value; finally, through updates of the target network and the main network, DQN can stably update the Q-value function, thereby learning the optimal policy:

$$\Delta w = \alpha(R_{t+1} + \gamma max_{\alpha}\widehat{q}(s_{t+1}, a_t, w) - \widehat{q}(s_t, s_t, w)) \cdot \nabla_w \widehat{q}(s_t, a_t, w) \quad (3)$$

In Equation (3), Δw represents the update amount of the parameters. $\nabla_w \hat{q}(s_t, a_t, w)$ is the gradient of the Q-value function with respect to the parameter w and is used to determine the update direction.

The DQN architecture has two neural nets, the Q network and the target network [36] and a component called the experience replay as shown in Figure 10. A deep neural network Q-network is used to approximate the state–action function [37]. When the agent interacts with the environment, each experience is stored in a replay buffer and later randomly sampled to train the Q network. The DQN is trained over many episodes with multiple steps, undergoing a series of operations in each time step.

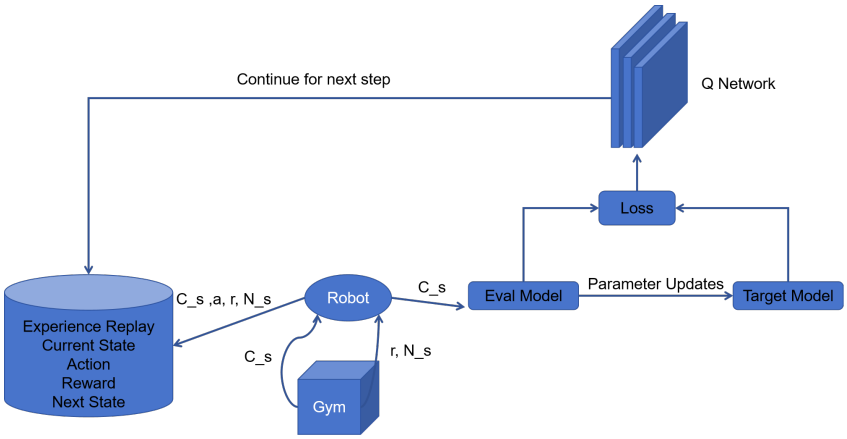

Figure 10. The framework of DQN.

First, the parameters of the Q network and the target network are randomly initialized. During the interaction with the environment, the agent collects experience data, and then selects actions using the ε-greedy policy. The experience replay executes the greedy action and receives the next state and reward [38]. It saves this observation as a sample of the training data and inputs it into both networks.

The Q network takes the current state and action from each data sample and predicts the Q value for that particular action as shown in Figure 11. This is the 'predicted Q value'.

The target network evaluates all possible actions for the next state and selects the action with the highest Q value to obtain the "best Q value" for that state. This best Q value is then multiplied by the discount factor γ to obtain the target Q value.

After training the DQN, the mean squared error loss is computed using the difference between the target Q value and the predicted Q value. Then, the loss is back-propagated to update the weights. After T steps of updating the target network and the main network, it can predict more accurate Q values.

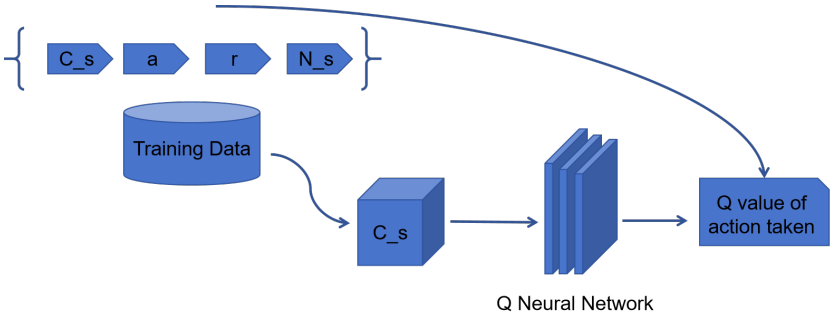

Figure 11. Q network predicts Q value.

3.2. The Re-DQN Algorithm

This paper proposes an improved DQN algorithm, called Re-DQN, to enhance the efficiency and stability of path planning. Complex terrain features are introduced in the environment design to simulate various obstacles and terrain variations. To strengthen the overall relevance of the model, a replay buffer is used to update the action values in each iteration, ultimately guiding the model's training and optimizing the decision-making process. The Re-DQN algorithm is as shown in Algorithm 1.

Algorithm 1 Re-DQN

Input : Replay buffer capacity B, discount factor γ, learning rate η, batch size b, target update frequency F_{tgt}, exploration rate ϵ, maximum episodes M_{ep}.
Output: Trained Q-network parameters θ.
Initialize Q-network $q(\theta)$ with global and local branches;
Initialize target network $q_{tgt}(\theta^{tgt}) \leftarrow q(\theta)$;
Initialize replay buffer $\mathcal{D}$ with capacity B;
Initialize optimizer for q using Adam with learning rate η;
for *episode* $= 1$ **to** M_{ep} **do**
  Reset environment, observe initial states s_g, s_l;
  Set *done* $\leftarrow$ *False*;
  while *not done* **do**
    Select the action a, to reach next state;
    Execute action a, observe reward r, next states s'_g, s'_l, and *done*;
    Store $(s_g, s_l, a, r, s'_g, s'_l, done)$ in $\mathcal{D}$;
    Update State: $s_g \leftarrow s'_g$, $s_l \leftarrow s'_l$;
    if $|\mathcal{D}| \geq b$ **then**
      Sample mini-batch $\{(s_g^i, s_l^i, a^i, r^i, s_g'^i, s_l'^i, d^i)\}_{i=1}^{b}$ from $\mathcal{D}$;
      Compute Target Q-values and Compute Loss $\mathcal{L}$;
      Train and update the weights of the Q-main network θ_{t+1};
      Zero gradients, update $\nabla_\theta \mathcal{L}$, update θ;
    end
  end
  if *episode* mod $F_{tgt} = 0$ **then**
    $\theta^{tgt} \leftarrow \theta$;
  end
end

3.2.1. Improvements Based on Action Selection

In complete coverage path planning, the robot selects appropriate actions in different states to achieve efficient coverage. To effectively balance exploration and exploitation, this study divides action selection into two parts: a dynamic ε-greedy strategy [39] combined with curiosity-driven exploration and a soft-max strategy [40].

The dynamic ε-greedy strategy and curiosity-driven exploration mechanism are used in combination. The core of the dynamic ε-greedy strategy lies in the gradual decay of the ε

value. Initially, ε is set to a high value to encourage the robot to explore more. As training progresses, the ε value gradually decreases, prompting the robot to rely more on the learned strategy. The specific formula is

$$\varepsilon = \varepsilon_{\text{end}} + (\varepsilon_{\text{start}} - \varepsilon_{\text{end}}) \times \exp\left(-\frac{S_{\text{done}}}{\varepsilon_{\text{decay}}}\right) \tag{4}$$

In Equation (4), S_{done} measures the training progress. At each step, a random number r is generated and compared to the current ε value. If $r > \varepsilon$, then the action with the highest current evaluation value is selected; otherwise, an action is chosen at random.

Building on this, a random noise proportional to ε is added to the state evaluation values as a curiosity bonus. This encourages the robot to explore states with higher uncertainty or novelty, preventing it from getting stuck in local optima and gradually shifting towards exploiting the learned strategy over time. This improves the efficiency and effectiveness of the complete coverage path planning.

Alongside the ε-greedy strategy, a soft-max strategy is also used, converting state evaluation values into a probability distribution for action sampling. Instead of relying on simple evaluation values, intrinsic curiosity-driven rewards are combined to enhance action selection. The curiosity bonus is added to the state evaluation values before applying the soft-max function. It can be computed based on the novelty of the state (such as visit frequency) or prediction error, encouraging the robot to explore novel and uncertain areas more effectively.

The action selection is then performed using the soft-max function:

$$q_i = \frac{exp(z_i)/\mathrm{T}}{\Sigma_j exp(z_j/\mathrm{T})} \tag{5}$$

where z_i represents the evaluation value of action i, and T is the temperature coefficient.

In addition to the ε-greedy strategy, we also use the soft-max strategy, which converts state evaluation values into a probability distribution through the soft-max function, from which actions are sampled. A temperature coefficient T is introduced to control the smoothness and diversity of the action selection [41,42]. The use of the temperature coefficient ensures more diverse action selection in the early stages of exploration, gradually converging to the optimal policy as training progresses. Typically, the soft-max strategy is used in scenarios that require more nuanced probability control, and the formula is as follows.

3.2.2. Reward Function

In CCPP, designing an appropriate reward function is crucial, as it guides the agent to learn suitable behaviors. It encourages the agent to cover uncovered areas quickly while minimizing redundant coverage of already covered regions [43]. Designing the reward function is a critical step, as it directly impacts the learning efficacy and task execution efficiency of the agent.

To calculate intrinsic rewards based on state novelty, it is suggested to use an exponential decay function based on visit frequency. This definition effectively encourages the robot to prioritize exploring unvisited or rarely visited areas while quickly reducing the reward after a state is repeatedly visited, thus avoiding the waste of exploration resources. The following is the definition of the intrinsic reward using an exponential decay function based on the state novelty:

$$r_{\text{intrinsic}}(s) = e^{-N(s)} \tag{6}$$

where $N(s)$ represents the visit frequency of state s, and the reward decays exponentially as the state is visited more frequently.

The reward function is defined as follows:

$$r = \begin{cases} -P_{\text{move}} & \\ r - P_{\text{obstacle}} & \textit{if collision = 1} \\ r + R_{\text{discover}} \times N_{\text{covered\_tiles}} & \textit{if } N_{\text{covered\_tiles}}\textit{=1 and collision = 0} \\ r + \left(\frac{N_{\text{count\_1}}}{N_{\text{count\_0}} + N_{\text{count\_0}}}\right) \times P_{\text{move}} & \textit{if } N_{\text{covered\_tiles}}\textit{=1 and collision = 0} \\ r - P_{\text{move}} & \textit{if } N_{\text{covered\_tiles}}\textit{=0 and collision = 0} \\ r + R_{\text{cc}} & \textit{if } C\textit{ = 1} \\ r - P_{\text{terrain}} \times \max(0, T_{\text{diff}}) & \textit{if } T_{\text{info}}\textit{=1} \\ r + \lambda \cdot r_{\text{intrinsic}} & \textit{if} N(s) \geq 0 \end{cases} \tag{7}$$

In the equation above, r is the reward, and P_{move} is a movement penalty for each step. In this way, the agent incurs a penalty for each step it takes, thereby encouraging it to choose shorter paths [44]. This approach ensures that the path length is effectively considered, rather than solely focusing on the destination of the path. C stands for "complete coverage completion". When C is 1, it means the agent has successfully covered all the accessible areas, and the task is complete. T_{info}=1 indicates that terrain information is present. T_{diff} represents the terrain difference between the current position and the target position. P_{obstacle} is a collision penalty, and R_{discover} is a reward for exploring new areas. R_{CC} is the reward given by the agent when they complete the map, and P_{terrain} is a penalty for terrain differences, while $N_{\text{covered\_tiles}}$ is the number of areas that are newly covered by the agent.

3.2.3. Improved DQN Network Architecture

In path planning using DQN [45,46], the number of obstacles in the environment is dynamically changing, which causes the input dimensionality to fluctuate. This, in turn, affects the stability of the DQN model. To overcome this challenge, a dynamic input structure is designed, which can adapt to changes in the number of obstacles in the environment while ensuring that the DQN model always processes input data with a fixed dimensionality during both training and inference.

Assuming that the system can handle a maximum of n obstacles, and the number of obstacles in the environment is m , the composition of the input vector $input$ depends on the number of obstacles. Below is a piecewise function that expresses how the input structure is processed, clearly outlining how the input dimensionality is handled under different obstacle counts:

$$\mathbf{input}(m) = \begin{cases} [\mathbf{o_a}, \mathbf{o_t}, \mathbf{o_1}, \mathbf{o_2}, \ldots, \ldots, \ldots, \ldots, \mathbf{o_n}], & \text{if } m \geq n \\ [\mathbf{o_a}, \mathbf{o_t}, \mathbf{o_1}, \mathbf{o_2}, \ldots, \mathbf{o_m}, \mathbf{0}, \mathbf{0}, \ldots, \mathbf{0}], & \text{if } m < n \end{cases} \tag{8}$$

The explanation is as follows:

- $\mathbf{o_a}$ represents the state of the robot (x, y, z, v_x, v_y, v_z).
- $\mathbf{o_t}$ represents the state of the target (x, y, z).
- $\mathbf{o_1}, \mathbf{o_2}, \mathbf{o_3}, \ldots\ldots \mathbf{o_n}$represent the states of the first n obstacles (x, y, z, v_x, v_y, v_z).

When $m = n$, the environment has exactly the maximum number of obstacles that our system can handle. In this case, the input vector $\mathbf{input}(m) = [\mathbf{o_a}, \mathbf{o_t}, \mathbf{o_1}, \mathbf{o_2}, \ldots, \mathbf{o_n}]$. When $m > n$, we have more obstacles in the environment than our system is designed to handle comprehensively. In this situation, we choose to still use the same input vector.

Zero Vector **0**: When the number of obstacles m in the environment is less than the maximum number of obstacles n set by the system, the zero vector **0** is used to fill the empty positions corresponding to the obstacle states in the input vector to ensure that the dimension of the input vector is always a fixed length related to n.

To improve the fitting of Q values, an incentive layer is added between the hidden layer and the output layer. This layer applies an incentive value to different Q values, enhancing the accuracy and effectiveness of the Q-value estimation. This allows the model to better adapt to environmental changes, thereby accelerating the training process and improving the model's performance.

In standard DQN, the Q-value update formula is

$$Q(s,a) = Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{9}$$

To encourage the agent to avoid obstacles, the dynamic incentive value can be adjusted based on the following two factors:

- Proximity : This function calculates the distance between the current action and the obstacles. The closer the action is to an obstacle, the lower the dynamic incentive value (i.e. negative incentive):

$$\text{Proximity}(s,a) = -\frac{1}{d(s,a)} \tag{10}$$

where $d(s,a)$ is the distance between the agent and the nearest obstacle in the new state s' after performing action a.

- Obstacl_Density: This function measures the density of obstacles in the environment. The higher the obstacle density, the more negative the incentive applied by the system, encouraging the agent to avoid areas with dense obstacles:

$$\text{Obstacle\_Density}(s) = \frac{\text{Number of obstacles in region}(s)}{\text{Area of region}(s)} \tag{11}$$

The formula for the dynamic incentive value Dynamic_Incentive(s,a) can be expressed as:

$$\text{Dynamic\_Incentive}(s,a) = \lambda_1 \cdot \text{Proximity}(s,a) + \lambda_2 \cdot \text{Obstacle\_Density}(s) \tag{12}$$

where:

- λ_1 and λ_2 are hyperparameters used to adjust the importance weights of Proximity and Obstacle_Density , respectively.

To further enhance exploration capabilities, a noisy-linear layer [47] is introduced, which injects random perturbations into the weights and biases of the network, improving the exploratory behavior of the model and the stability of training. Experiments demonstrate that the improved DQN significantly enhances the coverage rate and path planning efficiency, while also exhibiting stronger robustness and generalization in various environments. This design effectively leverages the complementarity of the global and local information, making the model better suited for complex and dynamic environments.

In the noisy-linear layer, we introduce noise to perturb the weights and biases. The formula is

$$\begin{aligned} W_{\text{noisy}} &= W_\mu + W_\sigma \odot \epsilon_W \\ b_{\text{noisy}} &= b_\mu + b_\sigma \odot \epsilon_b \end{aligned} \tag{13}$$

During each forward pass, the noise parameters are regenerated. The specific forward pass formula is

$$y = \left(W_\mu + W_\sigma \odot \epsilon_W\right)x + \left(b_\mu + b_\sigma \odot \epsilon_b\right) \tag{14}$$

3.2.4. Environmental Terrain Design

This study employs noise generation methods to create diverse and realistic terrain features. Due to the noise characteristics of the generator, the terrain maps may exhibit some random textures or fluctuations, represented by different grayscale regions. The terrain generation process includes the following steps:

(1) Noise Generation: The noise generator generates noise maps with specified dimensions and frequency. These noise maps serve as the foundation for creating terrain features.
(2) Normalization: The generated noise maps are normalized to ensure that the terrain values range from 0 (representing the lowest altitude) to 1 (representing the highest altitude). This normalization helps to evenly represent different terrain heights.

Perlin noise is a gradient noise function used to generate natural textures, and it is widely employed in computer graphics to create textures, terrains, and more. The generation of Perlin noise involves several steps, including gradient calculation and interpolation. Here are the basic formulas for Perlin noise:

$$\begin{aligned} \mathrm{PN}(x,y) = \mathrm{lerp}(&\mathrm{lerp}(\mathrm{dot}(i,j), \mathrm{dot}(i+1,j), s(x)), \\ &\mathrm{lerp}(\mathrm{dot}(i,j+1), \mathrm{dot}(i+1,j+1), s(x)), s(y)) \end{aligned} \tag{15}$$

where we have the following:

(1) $\mathtt{lerp}(a,b,t)$ represents a linear interpolation function: $\mathtt{lerp}(a,b,t) = a + t \times (b-a)$.
(2) $s(x)$ is the smooth interpolation function, typically using the cubic Hermite function:

$$s(x) = 3x^2 - 2x^3. \tag{16}$$

Perlin noise is often created by combining multiple layers of noise with different frequencies and amplitudes to achieve a more complex effect. This process is known as "Octaves":

$$\mathrm{PNO}(x,y) = \sum_{k=0}^{n-1}\left(\frac{1}{2^k} \times \mathrm{PN}\left(\frac{x}{2^k}, \frac{y}{2^k}\right)\right) \tag{17}$$

where n represents the number of layers (octaves).

In the experiment, we configure the terrain generator to generate terrain maps with varying sizes and frequencies. Specifically, we can set the map dimensions and adjust the frequency parameters to evaluate the impact of different levels of detail on the generated terrain.

As shown in Figures 12 and 13, grayscale values are used to represent the elevation of the terrain. Darker areas generally indicate lower terrain, while lighter areas indicate higher terrain. These grayscale values have been normalized to be displayed within an appropriate range in the images.

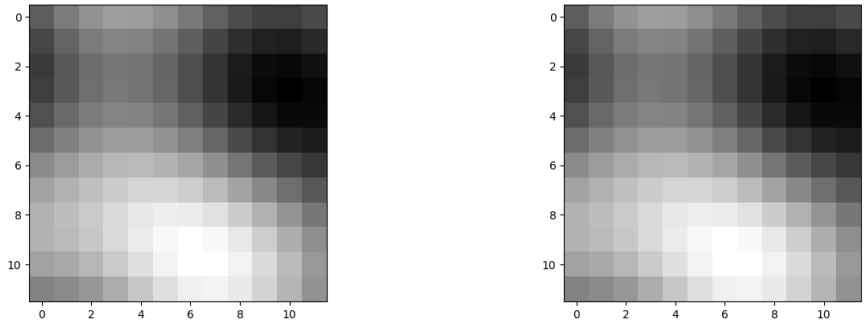

Figure 12. Ordinary-terrain map.

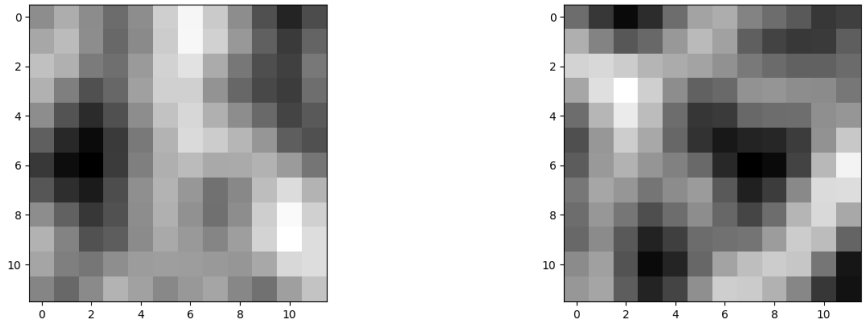

Figure 13. A map with a more complex terrain.

4. Simulation Results and Discussion

4.1. Setting of Simulation Conditions

To research and optimize the performance of the improved DQN algorithm for CCPP, simulations of the mobile robot's path planning on environmental maps have been conducted. The experimental objectives include the following:

(1) Validate the effectiveness of the improved DQN algorithm, i.e., verify the path coverage capability of the improved DQN in different environments through experiments, and assess its adaptability and performance in various complex scenarios.
(2) Evaluate the impact of key hyperparameters during training, that is, adjust hyperparameters such as the exploration rate, discount factor, and learning rate to analyze their effects on model training performance and stability.
(3) Enhance the model's adaptability in complex environments, i.e., test the model in environments with obstacles or irregular boundaries, study the performance of the DQN algorithm in such environments, and propose corresponding optimization strategies.

During the simulation experiment process, the basic environment parameters, rewards, and Re-DQN network model parameters are initialized first. Then, a preprocessed grid map, obstacle map, and coverage map are constructed, with the obstacle frequency, fill ratio, and height frequency set within the obstacle map.

Each time training begins, the lawnmower robot's position randomly appears on the grid map, and obstacles within the grid map change randomly. It is ensured that the lawnmower robot's initial position is not obstructed by obstacles. Subsequently, at each time step, the agent selects the next action based on the current state and path planning algorithm, updates the agent's position and map, and computes the immediate reward.

Termination conditions include the following:

(1) Coverage map indicating all reachable areas have been visited.
(2) Agent collides with an obstacle.
(3) Agent reaches the boundary of the map.

In summary, condition (1) represents a successful termination condition, while condition (2) represents a failure termination condition.

During training, each action generates a sample, stored in a replay memory pool.

In the CCPP based on deep reinforcement learning, my evaluation metrics mainly include three aspects: the average total nb_step, which measures the average number of actions taken by the agent to complete the task; the average total tiles_visited, which reflects the number of different areas covered by the agent during task execution; and the average total reward, which assesses the average feedback received by the agent during task execution. These metrics collectively evaluate the effectiveness of the path planning and the performance of the agent.

The simulation experiments in this study are run in an Ubuntu 20.04 environment, using Python as the development language. The parameters for the simulation platform are listed in Table 1.

Table 1. Simulation platform.

OS	Language	CPU	GPU	RAM
Ubuntu 22.04	Python 3.8	Intel i5-13400	RTX 4070 Ti	12 Gb

4.2. Outdoor Map Simulation Experiment

Figure 14 consists of three parts: the left side shows the terrain map; the middle displays the obstacle map, with static obstacles in black and dynamic obstacles in red; the right side shows the overlay of the terrain and obstacle maps, combining terrain height information with obstacle locations, illustrating the spatial relationship between the terrain and the obstacles.

We observe how agents navigate different types of terrain and how the complexity of the terrain affects their performance. The terrain influences the path planning methods in the following ways:

(1) Movement cost differences: Varying terrain heights result in different movement costs for agents, with the algorithm favoring paths with lower costs.
(2) Accessibility: Areas with significant elevation differences may be considered impassable, requiring the path planning algorithm to avoid these regions.
(3) Reward mechanism: The terrain information is integrated into the reward function, where larger elevation changes incur penalties, encouraging agents to select flatter paths.
(4) Environmental complexity: The terrain adds complexity to the planning process, requiring the algorithm to balance terrain difficulty with coverage efficiency.

Overall, the terrain affects the cost, accessibility, and efficiency of path planning. Preliminary results indicate that terrain maps with higher levels of detail provide a more challenging environment for the agents, thereby influencing their navigation and decision-making processes.

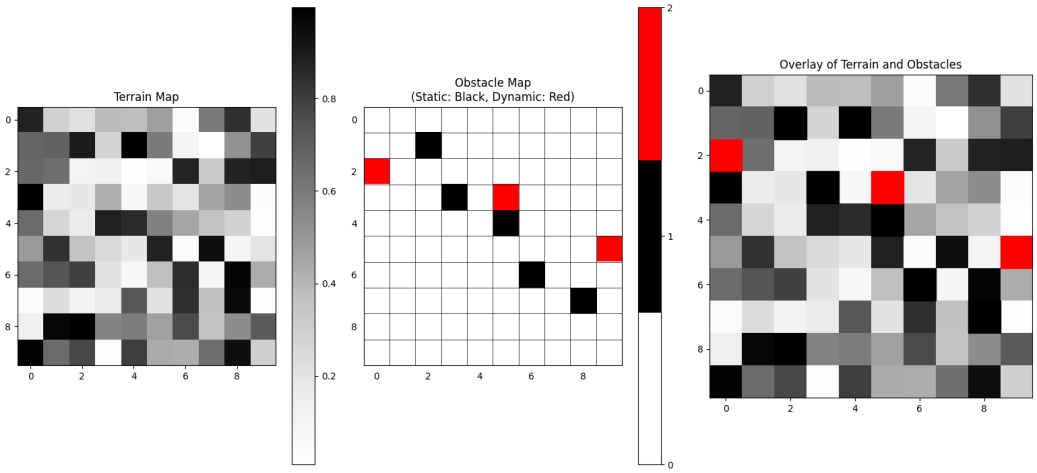

Figure 14. Overlay of terrain and obstacle distribution (10 × 10).The first figure is the terrain map, and the second figure is the obstacle map, where the black squares represent static obstacles and the red squares represent dynamic obstacles. The third figure shows the overlap of the terrain map and the obstacle map.

4.3. Parameter Analysis

First, set the size of the agent, ensuring that the size is greater than or equal to 1. Next, configure the agent's field of view and rotation capabilities. Before conducting experiments, it is essential to perform a validity check on the field of view. The field of view may not be set, in which case the agent might not have any vision constraints. If the field of view is to be set, it must be a positive odd integer greater than or equal to 1. This ensures that the field of view is symmetrically distributed around the agent, facilitating calculations and processing.

In this experiment, the settings for the modified DQN parameters need to be carefully considered in light of the environment, task nature, and resource constraints. Below is a detailed explanation and setting recommendations for each parameter:

(1) N: Determines the number of samples stored in the experience replay buffer. A buffer that is too small may lead to less diverse samples, affecting the model's generalization ability, while a buffer that is too large may increase memory demands. It is generally set between 5000 and 100,000. For simpler tasks, 5000 may be sufficient. For more complex tasks or if ample memory resources are available, a larger size can be chosen.

(2) γ: A larger discount factor means the model focuses more on long-term rewards, while a smaller factor means the model focuses more on short-term rewards. It is usually set between 0.9 and 0.99. A value of 0.9 indicates that the importance of future rewards gradually decreases, suitable for short-term decision tasks; 0.99 is more appropriate for tasks with a longer time span.

(3) ϵ: Initially, a high exploration rate helps in exploring new strategies; as training progresses, the exploration rate gradually decreases, leading to more reliance on the learned strategy.

- ϵ_{start}: 0.9 to 1.0. Usually set high to encourage more exploration at the beginning.
- ϵ_{end}: 0.01 to 0.1. A lower value ensures that the model relies more on the learned strategy during the later stages of training.
- ϵ_{decay}: 2000 to 10,000. A higher decay value means a longer exploration period, suitable for more complex tasks.

(4) τ: Frequency of updating the target network. A lower update frequency may lead to delayed target updates, while a higher frequency may cause instability in training. It

is recommended to set it between 500 and 5000 steps. For simpler tasks, 500 steps may suffice; for more complex tasks, a higher step count can be selected to stabilize training.

(5) α : Controls the step size of each parameter update. A learning rate that is too high may cause instability in training, while a learning rate that is too low may result in slow or stagnant training. It is usually advisable to start with a small learning rate, such as 0.001, and adjust based on the training outcomes.

(6) Environment and reward parameters: Additional settings related to the environment and rewards, which are not extensively discussed here, can be adjusted based on the size of your map and the terrain you wish to design. Relevant parameters include the following:

- P_{move} : 0.01–0.1.
- P_{terrain}: 0.01–0.1.
- P_{obstacle}: 0.1–1.0: adjusted according to the density of obstacles and the agent's ability to avoid them. A higher value should be set if you want the agent to be very sensitive to obstacles.
- P_{discover} and P_{coverage}: If the task focuses on coverage and discovering new areas, increase the values of P_{discover} and P_{coverage}. If the task emphasizes precise and efficient path planning, consider increasing P_{move} and P_{obstacle} values.

4.4. Complete Coverage Path Planning Results

First, a detailed comparison of the CCPP trajectories is conducted. In the first scenario, with a grid size of 16 × 16, the CPP trajectories for the simplest DQN-based algorithm, the improved DQN, are shown in Figures 15 and 16, respectively. It is evident that our algorithm has a clear advantage in CCPP compared to standard DQN.

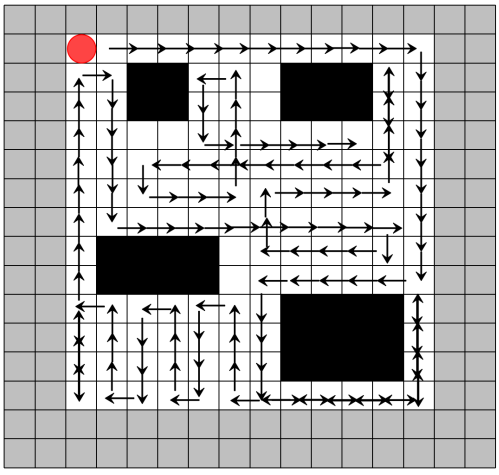

Figure 15. Coverage trajectory with DQN-based CCPP (16 × 16). The red circle represents the starting position, the gray border indicates the boundary of the map, and the mowing robot should not go beyond the map boundary. Black squares represent obstacles, and the arrows indicate the direction of movement.

Figure 17 shows the total reward variation across episodes during training in the CCPP task based on the Q-learning algorithm. The horizontal axis represents the number of training episodes, and the vertical axis represents the average total reward per episode.

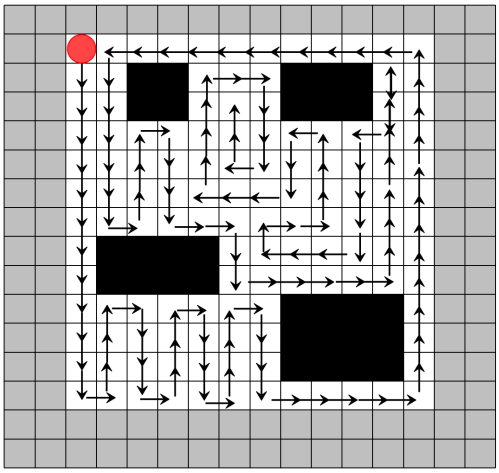

Figure 16. Coverage trajectory with Re-DQN-based CCPP (16 × 16). The red circle represents the starting position, Black squares represent obstacles, and the arrows indicate the direction of movement.

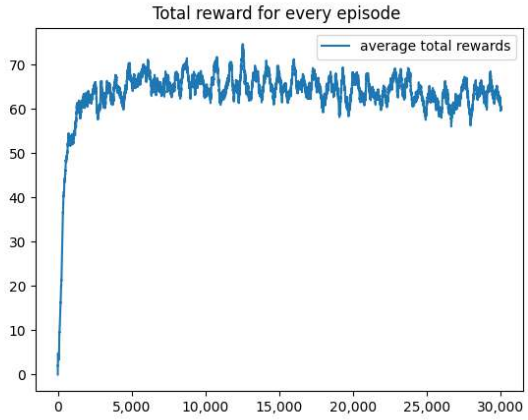

Figure 17. The Q-learning reward. The reward of Q-learning increases rapidly in the early stage, then fluctuates. The training process is unstable.

As shown in Figure 17, the reward of the Q-learning algorithm increases rapidly within the first 5000 episodes, but there are still significant fluctuations after reaching a certain level. This indicates that the model may exhibit some instability during the training process.

Additionally, the reward reaches a relatively high level in the early stages (around 3000 episodes) and does not show significant improvement afterward. This may indicate that the model's learning ability is limited and unable to further optimize its strategy, demonstrating the limited performance of Q-learning in the complex task of CCPP.

In path planning or coverage tasks, nb_step represents the number of steps the agent takes to complete the task. avg_nb_steps is the average number of steps over a recent period, reflecting the efficiency of the agent in executing the task.

If the value of avg_nb_steps decreases during training, it indicates that the agent is completing the task with fewer steps, which means it is becoming more efficient. The decreasing avg_nb_steps during training typically suggests that the agent's strategy is improving over time.

In Figure 18, it can be seen that DQN typically requires more steps, around 120, to complete the task throughout the training process. Re-DQN generally requires fewer steps in the same number of episodes, eventually stabilizing at around 100 steps. This indicates that Re-DQN performs better on this task as demonstrated by its lower average number of steps, and validates the importance of each component in improving the strategy.

In coverage tasks, avg_tiles_visited represents the average number of tiles covered by the agent in each episode. For coverage tasks, the goal is typically for the agent to cover as much of the area as possible. Therefore, the larger the avg_tiles_visited, the more extensive the area covered by the agent, indicating higher coverage efficiency. This suggests that the agent is continuously improving its strategy and is better able to complete the coverage task.

Figure 19 shows the average number of tiles covered by different algorithms in each episode in a CCPP task. Re-DQN covers more tiles per episode, around 108 tiles, while DQN covers about 87 tiles. This indicates that Re-DQN is more efficient in exploration and coverage, allowing it to cover a larger area in the same amount of time.

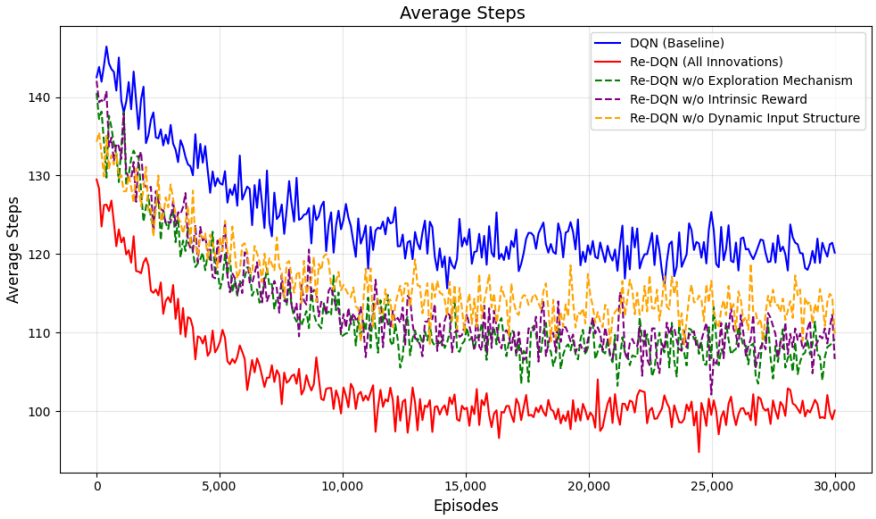

Figure 18. Average steps. It shows the variation in the average steps of different algorithms during the training process.

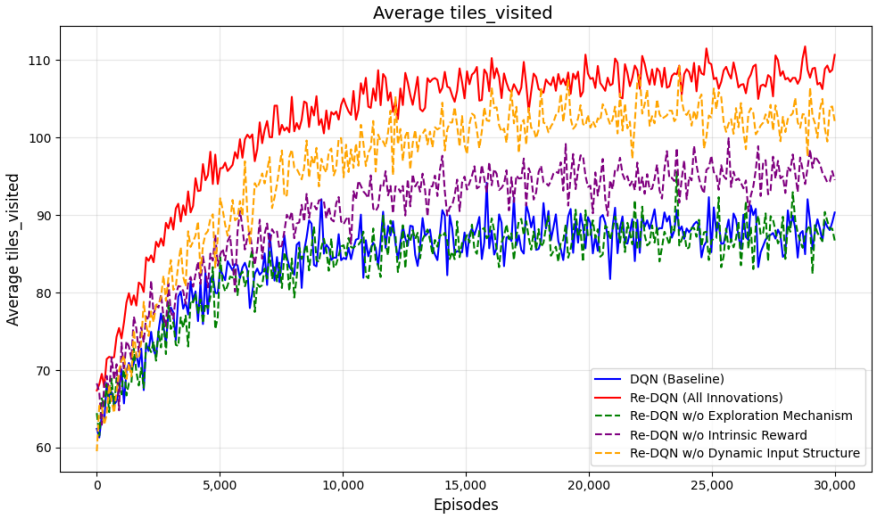

Figure 19. Average tiles_visited. It shows the changes in the average tiles visited of different algorithms during the training process.

Figure 20 illustrates the different performance of the models during the training process. The Re-DQN model stabilizes at a higher total reward level, around 85, indicating superior performance compared to DQN, which stabilizes at around 65.

In terms of convergence speed, the Re-DQN model improves rapidly, with total reward levels increasing rapidly, demonstrating faster learning capability. The DQN

model, however, has a slower overall improvement speed, and in the later stages, exhibits significant fluctuations, suggesting that the model's strategy has not yet fully stabilized.

In conclusion, Table 2 shows the comparison of different algorithms in the task, indicating that removing certain innovations may lead to a decrease in performance. On the other hand, Re-DQN achieves the best performance through its comprehensive optimization strategy.

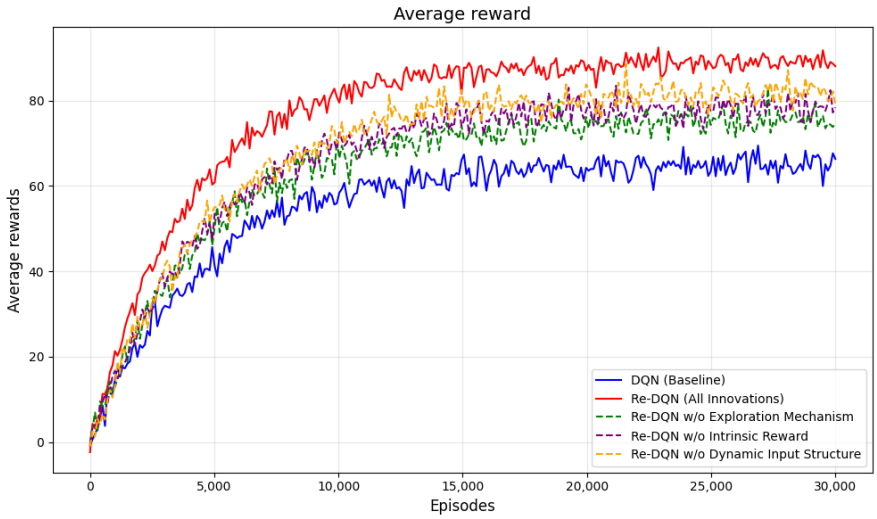

Figure 20. Average reward. It shows the changes in the average rewards of different algorithms, fully indicating that the Re-DQN algorithm performs better in this task.

Table 2. Algorithm comparison.

	DQN	**Re-DQN**	**Re-DQN w/o EM**	**Re-DQN w/o IR**	**Re-DQN w/o DIS**
Steps	120	100	108	110	115
Tiles visited	87	108	88	96	101
Rewards	65	85	72	79	81

Figure 21 displays obstacle maps under different fill ratios. The (a)–(c) represent the obstacle layouts for different fill ratios (0.04, 0.06, and 0.07).

(**a**) $fill\_ratio = 0.04$ (**b**) $fill\_ratio = 0.06$ (**c**) $fill\_ratio = 0.07$

Figure 21. Obstacle maps with different fill ratios. It shows three obstacle scenes with different fill ratios. Each scene contains black squares (static obstacles) and red squares (dynamic obstacles). With different fill ratios, the scenes and the number of obstacles are different.

From Table 3, we can see that the performance of each algorithm varies under different obstacle fill ratios. Re-DQN consistently performs as the strongest algorithm in all tests,

being able to adapt to environments with higher obstacle densities, maintaining a coverage rate ranging from 94% to 100%.

Table 3. Algorithm coverage efficiency under different $fill\_ratio$.

Algorithm	$fill\_ratio$ = 0.04	$fill\_ratio$ = 0.06	$fill\_ratio$ = 0.07
Boustrophedon	92%	89%	86%
A* Coverage Algorithm	95%	94%	93%
DQN	87%	83%	78%
DDQN	89%	83%	82%
Dueling DQN	87%	84%	81%
PPO	95%	92%	90%
Re-DQN (Our algorithm)	100%	97%	94%

Figure 22 depicts varying levels of terrain complexity, including simple flat, moderately complex, and highly complex environments, to evaluate the algorithm's performance under different conditions.

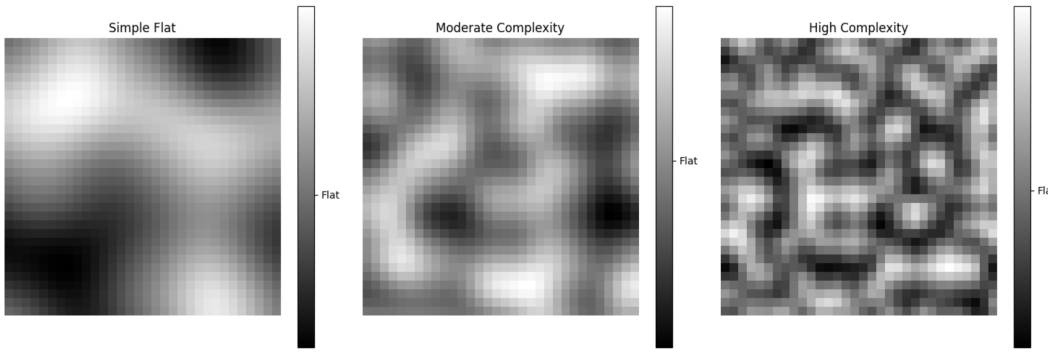

Figure 22. Varying terrain complexity. It shows three terrain images with different levels of complexity. The gradient bar displays color changes from light to dark. Larger color changes indicate more complex terrain.

Based on Table 4, we can conclude that in complex environments, Re-DQN, compared to the other path planning algorithms, demonstrates higher flexibility and robustness, making it more suitable for more complex and diverse geographical environments.

Table 4. Algorithm coverage efficiency under different terrain complexities.

Algorithm	Simple Flat	Moderate Complexity	High Complexity
Boustrophedon	92%	83%	78%
A* Coverage Algorithm	96%	91%	83%
DQN	84%	83%	78%
DDQN	86%	83%	80%
Dueling DQN	87%	84%	81%
PPO	95%	92%	86%
Re-DQN (Our algorithm)	100%	95%	93%

Table 5 systematically compares the performance of traditional algorithms and reinforcement learning algorithms in coverage tasks through multiple metrics, including path length, coverage rate, and path redundancy rate. Re-DQN demonstrates its superiority with shorter paths, lower redundancy rates, and higher adaptability scores, while maintaining moderate complexity. This highlights Re-DQN as an efficient and balanced solution for coverage tasks.

Table 5. Comparison of various algorithms for coverage tasks.

Algorithm	Path Length	Coverage (%)	Redundancy (%)	Adaptability (%)	Complexity
Boustrophedon	237	87	18.4	60	Low
A* Coverage Algorithm	212	95	32.7	65	Low
DQN	189	82	28.2	65	Moderate
DDQN	178	84	27.4	75	Moderate
Dueling DQN	183	81	26.3	65	Moderate
PPO	173	93	11.4	85	very high
Re-DQN(Our algorithm)	159	95	6.2	90	Moderate

5. Conclusions

5.1. Main Conclusions and Findings

This study proposed a novel algorithm, Re-DQN, for complete coverage path planning in lawn mowing robots using deep reinforcement learning. Taking into account the limitations of traditional algorithms, Re-DQN improved tiles_visited by 24.14%, increased the reward level by approximately 30.76%, and increased step efficiency by 16.67%.

By modeling the environment and dynamically adjusting the grid map resolution based on the robot's size, the algorithm introduces a new exploration mechanism for action selection, significantly enhancing the robot's ability to explore new areas and optimize paths, thus effectively avoiding local optima. A dynamic input structure is incorporated to address the challenges posed by changes in the number of obstacles in the environment. This improvement enables the robot to better navigate high-dimensional continuous state spaces, ensuring more comprehensive coverage and reducing planning time. The carefully designed reward function encourages the robot to cover new areas while minimizing redundant coverage, penalties for collisions, and terrain differences, leading to more efficient path planning.

5.2. Main Limitation of the Research

In the modeling phase, the actual motion characteristics of the lawn mower robot were not fully considered. The real-world model is much more complex than the one used in this study. Therefore, this complete coverage path planning algorithm may perform poorly in certain situations. Despite optimization of the exploration strategy, the algorithm may still fall into local optima, especially in large-scale or diverse environments. Finally, while the current algorithm performs well in single-robot systems, its scalability and coordination in multi-robot systems require further research and optimization.

5.3. Future Research Prospects

To further enhance its practical value and performance [48], future work will focus on studying the adaptability of Re-DQN in complex environments. This will involve testing and optimizing the Re-DQN algorithm in more complex and dynamically changing environments, including handling different types of environmental conditions. Additionally, we will explore the potential of extending the Re-DQN algorithm to multi-robot systems, investigating collaboration strategies and communication mechanisms between robots to achieve more efficient and large-scale complete coverage path planning.

Author Contributions: Conceptualization, Z.-M.L.; methodology, Y.C.; software, Y.C.; validation, H.L.; formal analysis, Y.-M.Z.; investigation, Y.C.; resources, J.-L.C.; data curation, Y.-M.Z.; writing—original draft preparation, Y.C.; writing—review and editing, H.L. and Z.-M.L.; visualization, J.-L.C.; supervision, Z.-M.L.; project administration, Z.-M.L.; funding acquisition, Z.-M.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research is partially supported by Special Fund of Huanjiang Laboratory and Ningbo Science and Technology Innovation 2025 major project under grants 2023Z040.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The study did not report any data.

Acknowledgments: We would like to thank our students for their work during the algorithm testing.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Choset, H. Coverage for robotics–a survey of recent results. *Ann. Math. Artif. Intell.* **2001**, *31*, 113–126. [CrossRef]
2. Latombe, J.-C.; Latombe, J.-C. Exact cell decomposition. In *Robot Motion Planning*; Springer Science & Business Media: Berlin, Germany, 1991; pp. 200–247.
3. Oksanen, T.; Visala, A. Coverage path planning algorithms for agricultural field machines. *J. Field Robot.* **2009**, *26*, 651–668. [CrossRef]
4. Choset, H.; Pignon, P. Coverage path planning: The boustrophedon cellular decomposition. In *Field and Service Robotics*; Springer: Berlin/Heidelberg, Germany, 1998; pp. 203–209.
5. Choset, H. Coverage of known spaces: The boustrophedon cellular decomposition. *Auton. Robot.* **2000**, *9*, 247–253. [CrossRef]
6. Huang, W.H. Optimal line-sweep-based decompositions for coverage algorithms. In Proceedings of the 2001 ICRA, IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164), Seoul, Republic of Korea, 21–26 May 2001; IEEE: Piscataway, NJ, USA, 2001; Volume 1, pp. 27–32.
7. Moravec, H.; Elfes, A. High resolution maps from wide angle sonar. In Proceedings of the 1985 IEEE International Conference on Robotics and Automation, St. Louis, MO, USA, 25–28 March 1985; IEEE: Piscataway, NJ, USA, 1985; Volume 2 pp. 116–121.
8. Hodgkin, A.L.; Huxley, A.F. A quantitative description of membrane current and its application to conduction and excitation in nerve. *Bull. Math. Biol.* **1990**, *52*, 25–71. [CrossRef]
9. Grossberg, S. Nonlinear neural networks: Principles, mechanisms, and architectures. *Neural Netw.* **1988**, *1*, 17–61. [CrossRef]
10. Gabriely, Y.; Rimon, E. Competitive on-line coverage of grid environments by a mobile robot. *Comput. Geom.* **2003**, *24*, 197–224. [CrossRef]
11. Gonzalez, E.; Alarcon, M.; Aristizabal, P.; Parra, C. Bsa: A coverage algorithm. In Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No. 03CH37453), Las Vegas, NV, USA, 27–31 October 2003; IEEE: Piscataway, NJ, USA, 2003; Volume 2, pp. 1679–1684.
12. Choi, Y.-H.; Lee, T.-K.; Baek, S.-H.; Oh, S.-Y. Online complete coverage path planning for mobile robots based on linked spiral paths using constrained inverse distance transform. In Proceedings of the 2009 IEEE/RSJ International Conference on Intelligent Robots and Systems, St. Louis, MO, USA, 10–15 October 2009; IEEE: Piscataway, NJ, USA, 2009; pp. 5788–5793.
13. Luo, C.; Yang, S.X.; Stacey, D.A.; Jofriet, J.C. A solution to vicinity problem of obstacles in complete coverage path planning. In Proceedings of the 2002 IEEE International Conference on Robotics and Automation (Cat. No. 02CH37292), Washington, DC, USA, 11–15 May 2002; IEEE: Piscataway, NJ, USA, 2002; Volume 1, pp. 612–617.
14. Yang, S.X.; Luo, C. A neural network approach to complete coverage path planning. *IEEE Trans. Syst. Man Cybern. Part B (Cybernetics)* **2004**, *34*, 718–724. [CrossRef]
15. Tang, G.; Tang, C.; Zhou, H.; Claramunt, C.; Men, S. R-DFS: A coverage path planning approach based on region optimal decomposition. *Remote Sens.* **2021**, *13*, 1525. [CrossRef]
16. Zhang, J.; Zhang, J.; Ma, Z.; He, Z. Using partial-policy q-learning to plan path for robot navigation in unknown enviroment. In Proceedings of the 2017 10th International Symposium on Computational Intelligence and Design (ISCID), Hangzhou, China, 9–10 December 2017; IEEE: New York, NY, USA, 2017; Volume 1, pp. 192–196.
17. Szczerba, R.J.; Galkowski, P.; Glicktein, I.S.; Ternullo, N. Robust algorithm for real-time route planning. *IEEE Trans. Aerosp. Electron. Syst.* **2000**, *36*, 869–878. [CrossRef]
18. LaValle, S. Rapidly-exploring random trees: A new tool for path planning. In *Research Report 9811*; 1998. Available online: https://msl.cs.illinois.edu/~lavalle/papers/Lav98c.pdf (accessed on 7 January 2025).
19. Ren, S.; He, K.; Girshick, R.; Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. *Adv. Neural Inf. Process. Syst.* **2015**, *28*, 91–99. [CrossRef]
20. Tai, L.; Paolo, G.; Liu, M. Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation. In Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vancouver, BC, Canada, 24–28 September 2017; pp. 31–36.

21. Wu, J.; Yu, P.; Feng, L.; Zhou, F.; Li, W.; Qiu, X. 3D aerial base station position planning based on deep Q-network for capacity enhancement. In Proceedings of the 2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), Arlington, VA, USA, 8–12 April 2019; IEEE: New York, NY, USA, 2019; pp. 482–487.
22. Zhou, S.; Liu, X.; Xu, Y.; Guo, J. A deep Q-network (DQN) based path planning method for mobile robots. In Proceedings of the 2018 IEEE International Conference on Information and Automation (ICIA), Wuyishan, China, 11–13 August 2018; IEEE: New York, NY, USA, 2018; pp. 366–371.
23. Van Hasselt, H.; Guez, A.; Silver, D. Deep reinforcement learning with double q-learning. In Proceedings of the AAAI Conference on Artificial Intelligence, Phoenix, AZ, USA, 12–17 February 2016; Volume 30.
24. Zhang, F.; Gu, C.; Yang, F. An improved algorithm of robot path planning in complex environment based on Double DQN. In *Advances in Guidance, Navigation and Control, Proceedings of the 2020 International Conference on Guidance, Navigation and Control, ICGNC 2020, Tianjin, China, 23–25 October 2020*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 303–313.
25. Lei, X.; Zhang, Z.; Dong, P. Dynamic Path Planning of Unknown Environment Based on Deep Reinforcement Learning. *J. Robot.* **2018**, *2018*, 5781591. [CrossRef]
26. Sonny, A.; Yeduri, S.R.; Cenkeramaddi, L.R. Q-learning-based unmanned aerial vehicle path planning with dynamic obstacle avoidance. *Appl. Soft Comput.* **2023**, *147*, 110773. [CrossRef]
27. Yang, X.; Han, Q. Improved DQN for Dynamic Obstacle Avoidance and Ship Path Planning. *Algorithms* **2023**, *16*, 220. [CrossRef]
28. Wang, T.; Peng, X.; Wang, T.; Liu, T.; Xu, D. Automated design of action advising trigger conditions for multiagent reinforcement learning: A genetic programming-based approach. *Swarm Evol. Comput.* **2024**, *85*, 101475. [CrossRef]
29. Yuan, G.; Xiao, J.; He, J.; Jia, H.; Wang, Y.; Wang, Z. Multi-agent cooperative area coverage: A two-stage planning approach based on reinforcement learning. *Inf. Sci.* **2024**, *678*, 121025. [CrossRef]
30. Ramezani, M.; Amiri Atashgah, M.A.; Rezaee, A. A Fault-Tolerant Multi-Agent Reinforcement Learning Framework for Unmanned Aerial Vehicles–Unmanned Ground Vehicle Coverage Path Planning. *Drones* **2024**, *8*, 537. [CrossRef]
31. Sanchez-Ibanez, J.R.; Pérez-del Pulgar, C.J.; García-Cerezo, A. Path planning for autonomous mobile robots: A review. *Sensors* **2021**, *21*, 7898. [CrossRef]
32. Borenstein, J. Vfh+: Reliable obstacle avoidance for fast mobile robots. In Proceedings of the 1998 IEEE International Conference on Robotics and Automation (Cat. No. 98CH36146), Leuven, Belgium, 20–20 May 1998.
33. Lin, Y.Y.; Ni, C.C.; Lei, N.; Gu, X.D.; Gao, J. Robot coverage path planning for general surfaces using quadratic differentials. *arXiv* **2017**, arXiv:1701.07549.
34. Dam, T.; Chalvatzaki, G.; Peters, J.; Pajarinen, J. Monte-carlo robot path planning. *IEEE Robot. Autom. Lett.* **2022**, *7*, 11213–11220. [CrossRef]
35. Watkins, C.; Dayan, P. Technical Note: Q-Learning. *Mach. Learn.* **2004**, *8*, 279–292. [CrossRef]
36. Zhang, J.; Zhang, C.; Chien, W.C. Overview of deep reinforcement learning improvements and applications. *J. Internet Technol.* **2021**, *22*, 239–255.
37. Kiran, B.R.; Sobh, I.; Talpaert, V.; Mannion, P.; Al Sallab, A.A.; Yogamani, S.; Pérez, P. Deep reinforcement learning for autonomous driving: A survey. *IEEE Trans. Intell. Transp. Syst.* **2021**, *23*, 4909–4926. [CrossRef]
38. Kim, M.J.; Park, H.; Ahn, C.W. Nondominated policy-guided learning in multi-objective reinforcement learning. *Electronics* **2022**, *11*, 1069. [CrossRef]
39. Mignon, A.d.S.; da Rocha, R.L.d.A. An adaptive implementation of ϵ-greedy in reinforcement learning. *Procedia Comput. Sci.* **2017**, *109*, 1146–1151. [CrossRef]
40. Li, J.; Shi, X.; Li, J.; Zhang, X.; Wang, J. Random curiosity-driven exploration in deep reinforcement learning. *Neurocomputing* **2020**, *418*, 139–147. [CrossRef]
41. He, Y.L.; Zhang, X.L.; Ao, W.; Huang, J.Z. Determining the optimal temperature parameter for Softmax function in reinforcement learning. *Appl. Soft Comput.* **2018**, *70*, 80–85. [CrossRef]
42. Pan, L.; Rashid, T.; Peng, B.; Huang, L.; Whiteson, S. Regularized softmax deep multi-agent q-learning. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 1365–1377.
43. Wang, Y.; He, Z.; Cao, D.; Ma, L.; Li, K.; Jia, L.; Cui, Y. Coverage path planning for kiwifruit picking robots based on deep reinforcement learning. *Comput. Electron. Agric.* **2023**, *205*, 107593. [CrossRef]
44. Guo, S.; Zhang, X.; Du, Y.; Zheng, Y.; Cao, Z. Path planning of coastal ships based on optimized DQN reward function. *J. Mar. Sci. Eng.* **2021**, *9*, 210. [CrossRef]
45. Zhu, S.; Gui, L.; Cheng, N.; Sun, F.; Zhang, Q. Joint design of access point selection and path planning for UAV-assisted cellular networks. *IEEE Internet Things J.* **2019**, *7*, 220–233. [CrossRef]
46. Yang, Y.; Juntao, L.; Lingling, P. Multi-robot path planning based on a deep reinforcement learning DQN algorithm. *CAAI Trans. Intell. Technol.* **2020**, *5*, 177–183. [CrossRef]

47. Meng, L.; Goodwin, M.; Yazidi, A.; Engelstad, P.E. Improving the diversity of bootstrapped dqn by replacing priors with noise. *IEEE Trans. Games* **2022**, *15*, 580–589. [CrossRef]
48. Xing, B.; Wang, X.; Yang, L.; Liu, Z.; Wu, Q. An algorithm of complete coverage path planning for unmanned surface vehicle based on reinforcement learning. *J. Mar. Sci. Eng.* **2023**, *11*, 645. [CrossRef]